STUDENT ATTENDANCE MANAGEMENT SYSTEM USING DEVOPS TOOLS

PROJECT REPORT

Submitted by: _____

Department: Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Institution: _____

Project Repository

https://github.com/Ksiri03/student-attende

Abstract

The Student Attendance Management System is a web-based application intended to simplify daily attendance recording and monthly attendance calculations. Faculty can view registered students, record attendance for seven periods per day, retrieve date-wise records, and review monthly summaries. The backend uses Node.js 22 and Express, with SQLite for persistent data storage. The project demonstrates DevOps practices through Git for version control, Jenkins for continuous integration, Docker for containerization, Kubernetes for orchestration, and Selenium WebDriver with JavaScript for automated browser checks.

1. Introduction

Manual attendance tracking can be time-consuming when a class contains many students and attendance must be recorded for several periods each day. The application centralizes attendance operations in a web interface and provides APIs for student records, daily attendance, attendance history, monthly reports, and CSV export. It is packaged as a Docker image, validated by Jenkins, and deployed with Docker Compose and Kubernetes.

2. Problem Statement

Manual attendance records require repeated data entry and separate calculations. This can lead to errors, difficulty retrieving historical records, and delays in identifying students with low attendance. A digital system is required to store records by student, date, and period and calculate monthly summaries consistently.

3. Objectives

- Manage registered students in a centralized class list.
- Record Present or Absent status for seven periods per day.
- Store attendance by student, date, and period with a uniqueness constraint.
- Calculate working periods, present periods, absent periods, and attendance percentage.
- Provide student-wise reports, class-wise monthly summaries, daily history, and CSV export.
- Identify students below the configured attendance threshold (75% by default).
- Demonstrate version control, CI, containerization, orchestration, and automated checks.

4. Scope and Functional Modules

Student Management

Add students using roll number, name, and department; list, edit, and delete records. Roll numbers are unique and duplicates are rejected.

Daily Attendance

Select a date, display the class list, and mark attendance across Periods 1–7. Working periods can be toggled for the selected day, and the register can be saved, reloaded, and corrected.

Attendance Record Management

Save and retrieve records by date and period. A unique key prevents duplicate student/date/period entries, and deleting a student cascades to their attendance.

Monthly Attendance Calculation

Calculate working periods, present and absent totals, attendance percentage, and low-attendance status against a configurable threshold.

Reports

Retrieve monthly class reports, individual student reports with daily breakdowns, daily history, and CSV exports. Settings configures the threshold (0–100).

5. Technologies Used

Technology area	Tool / implementation
Frontend	HTML5, CSS3, and vanilla JavaScript single-page application
Backend	Node.js 22 with Express.js
Database	SQLite using better-sqlite3
Version control	Git repository on GitHub (main branch)
Continuous integration	Jenkins Pipeline (Dockerized controller and agent)
Containerization	Docker multi-stage image
Orchestration / deployment	Kubernetes (kind cluster) with a NodePort Service
Automated browser checks	Selenium WebDriver for JavaScript with headless Chrome
Runtime endpoints	App 3000 · Jenkins 8080 · Kubernetes 30080

6. System Architecture

A user interacts with the browser-based frontend. The frontend communicates with the Express application, which validates requests and reads and writes attendance data in SQLite. Source code is maintained in a Git repository. Jenkins checks out the repository, verifies the Node.js/npm toolchain, installs dependencies, performs syntax checks, runs 22 API tests, builds a Docker image, and runs a container smoke test. Kubernetes runs the container through a Deployment and exposes it using a NodePort Service on port 30080. Selenium drives Chrome to verify the main application workflows.

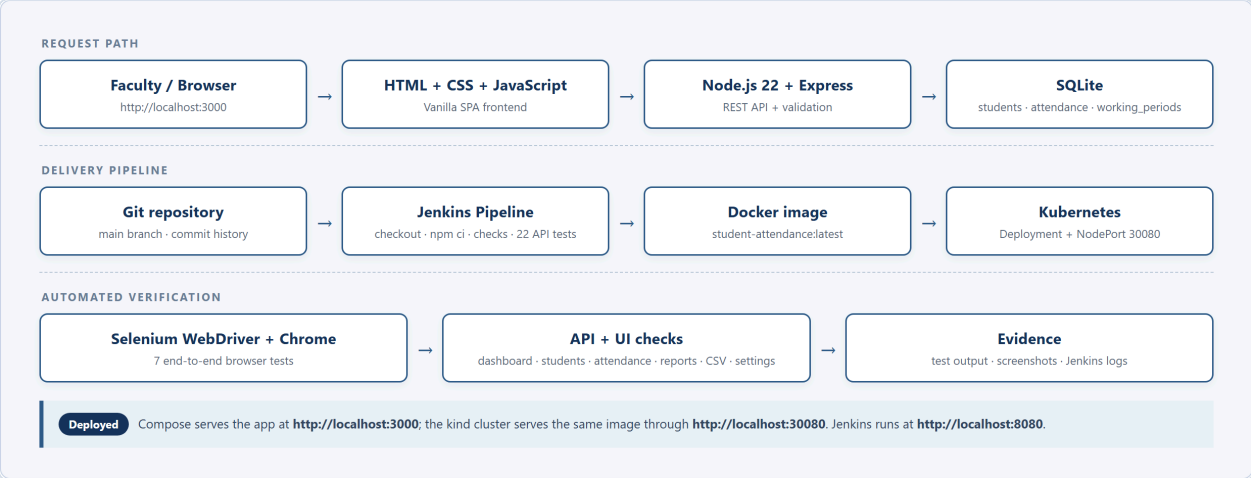


Figure 1 — Application request path, delivery pipeline, and automated verification.

22 / 22 API tests passing	**7 / 7** Selenium tests passing	**12** smoke checks passing	**62 / 62** adversarial API checks

9. DevOps Implementation

9.1 Git and GitHub

The project source is maintained in a public Git repository on GitHub at the URL shown below. The repository contains the application source, Dockerfile, Jenkinsfile, Kubernetes manifests, automated tests, and documentation. The final commit built for this report is **dbabc94** (“Point project and Jenkins pipeline at GitHub repository”), which follows **dcf334d** (“Disable Jenkins authentication for local demo”) and **122609d** (“Stabilize no-data Selenium test waits”). The main branch holds the complete project history, and Jenkins clones this repository for every build.

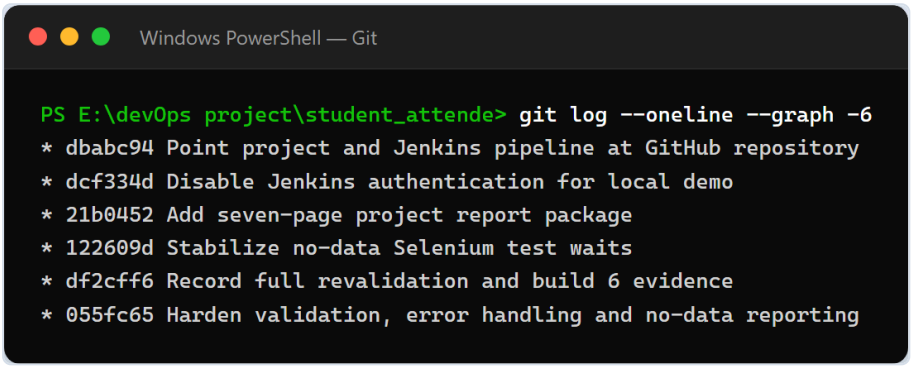


Figure 2 — Local Git history on the main branch.

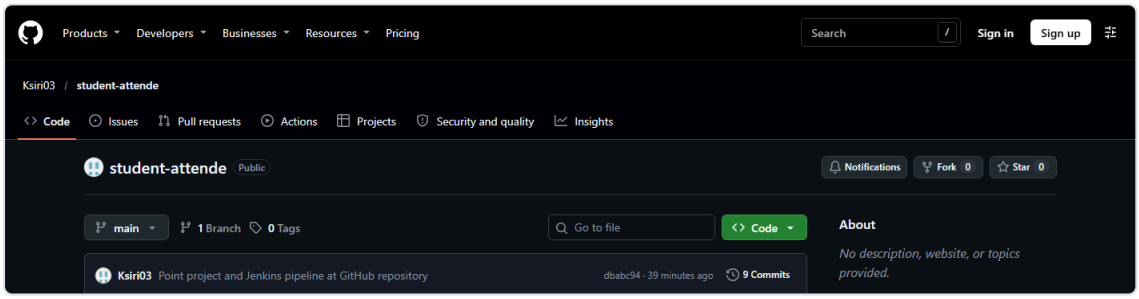


Figure 3 — Public GitHub repository at github.com/Ksiri03/student-attende.

9.2 Jenkins Pipeline

A Jenkins Pipeline job named **student-attendance-pipeline** retrieves the Jenkinsfile directly from the GitHub repository. The pipeline verifies the environment, installs dependencies with `npm ci`, runs syntax checks and all 22 API tests, builds the Docker image **student-attendance:ci-<build>**, retags it as **student-attendance:latest**, and starts the container for a smoke test. Build **#14** cloned commit dbabc94 from GitHub and completed successfully in **19.7 seconds**. The pipeline reported Node.js v22.23.4, npm 10.9.9, Git 2.47.3, and Docker CLI 27.5.1.

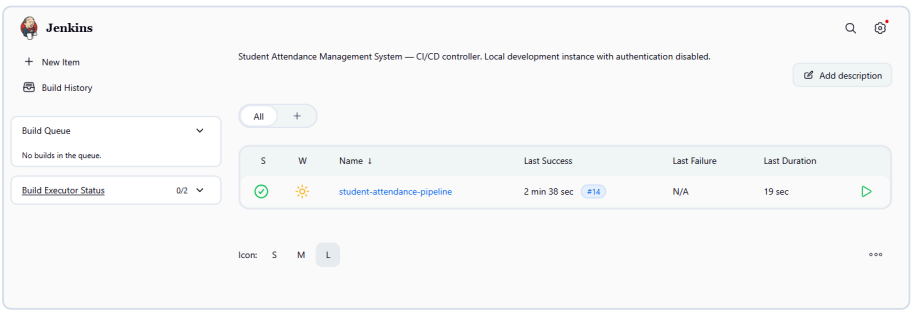


Figure 4 — Jenkins dashboard with the pipeline job.

#14	19.7 s	7 / 7	22 / 22
successful build	total duration	stages passed	API tests

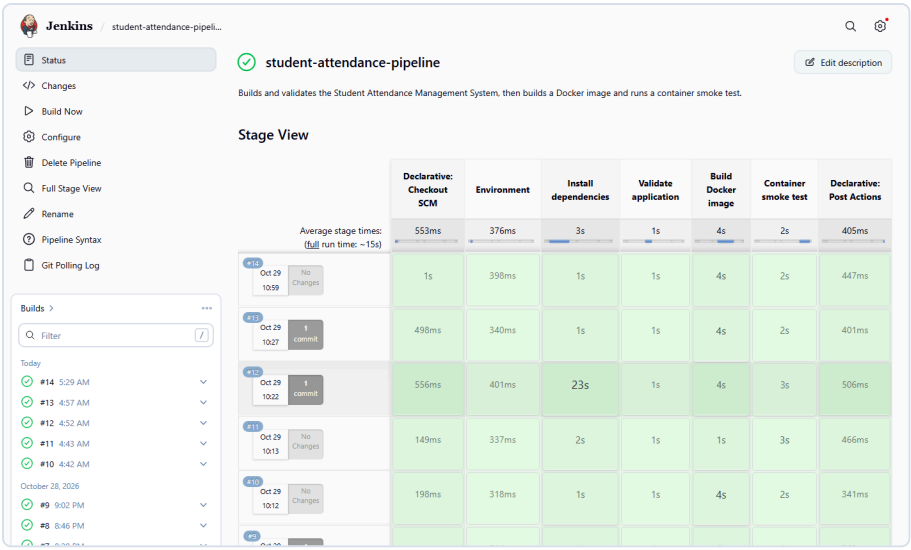


Figure 5 — Build #14 stage view: every pipeline stage passed.

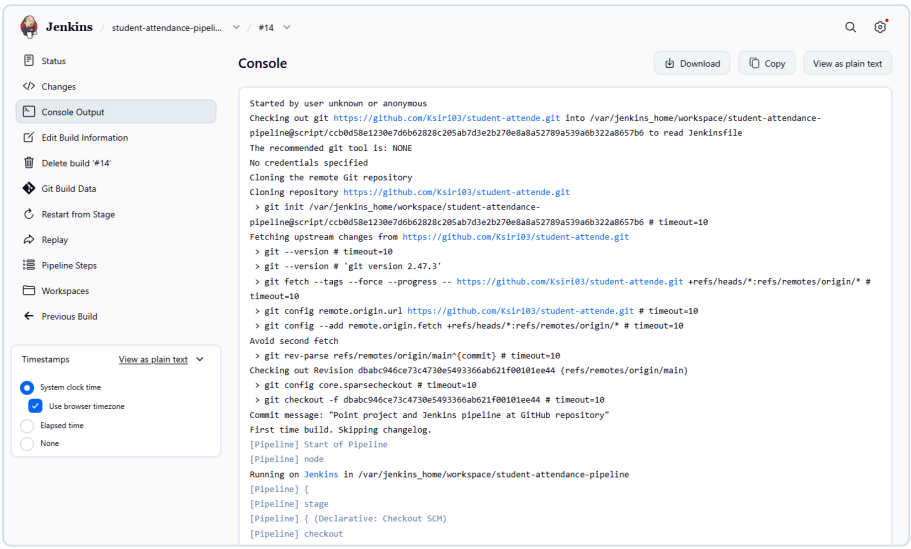


Figure 6 — Build #14 console output: GitHub checkout, dependency install, tests, image build, and smoke test.

9.3 Docker

A multi-stage Dockerfile packages the Node.js application and its dependencies. The dependency stage uses `node:22-bookworm-slim` and installs Python 3, make, and g++ so the native better-sqlite3 module can be compiled when a prebuilt binary is unavailable. The runtime stage copies only the production dependencies and application files, runs as the non-root node user, exposes port 3000, includes a health check against `/api/health`, and starts `server.js`. The final image is tagged **student-attendance:latest** (image ID **4d6a62b8003f**). The temporary CI tag was removed after deployment, so only the working image remains, alongside the Jenkins controller and the kind node image required by the local infrastructure.



Figure 7 — Docker images after cleanup: the working latest image, the Jenkins controller, and the kind node image.

9.4 Kubernetes

The Kubernetes Deployment is named **student-attendance** and runs one replica of the `student-attendance:latest` container on port 3000. The **student-attendance** Service is a NodePort Service exposed on port **30080**. A 1 Gi PersistentVolumeClaim named **attendance-data** stores the SQLite database, so data survives pod replacement. The manifests are applied with Kustomize to the `student-attendance` namespace. The deployment was verified at **http://localhost:30080**, and the same report totals were served after the pod was deleted and recreated.

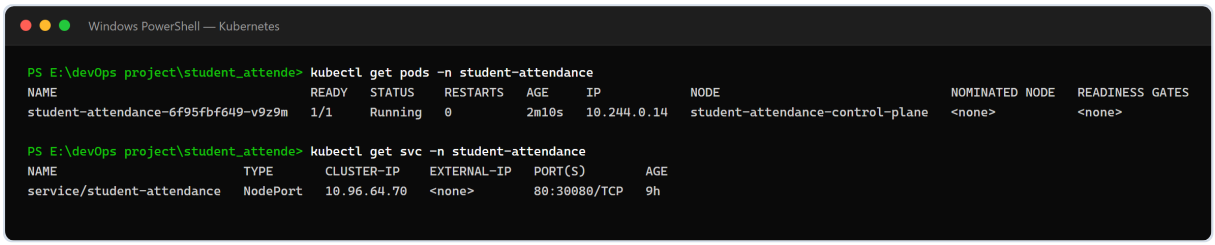


Figure 8 — Running pod and NodePort Service in the student-attendance namespace.

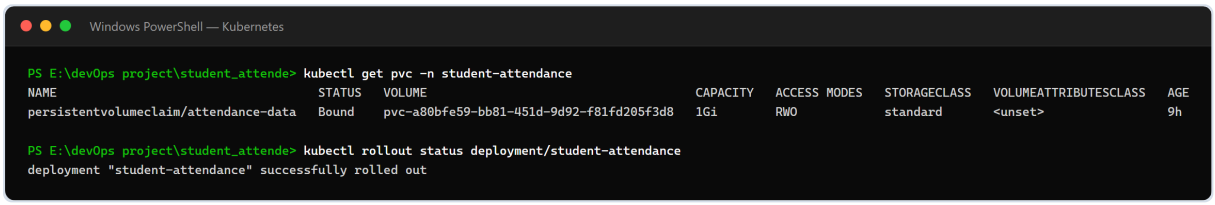


Figure 9 — Bound PersistentVolumeClaim and successful Deployment rollout.

9.5 Selenium Automated Checks

Selenium WebDriver is installed as a development dependency, and a JavaScript test suite is run with headless Chrome. The suite starts its own application instance with a temporary SQLite database, so it does not modify demo data. All **seven** tests pass. The checks cover dashboard statistics, adding a student through the interface, marking and saving all seven periods, monthly and individual reports, CSV export, the no-data state for students without attendance, and changing the attendance threshold.

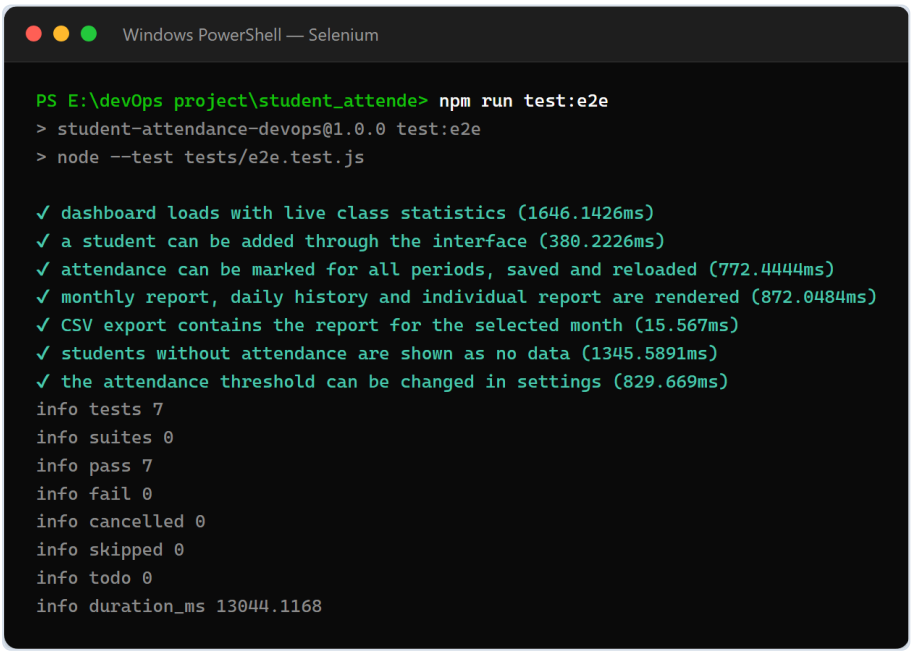


Figure 10 — Selenium end-to-end suite: 7 of 7 tests passed.

Application Screenshots

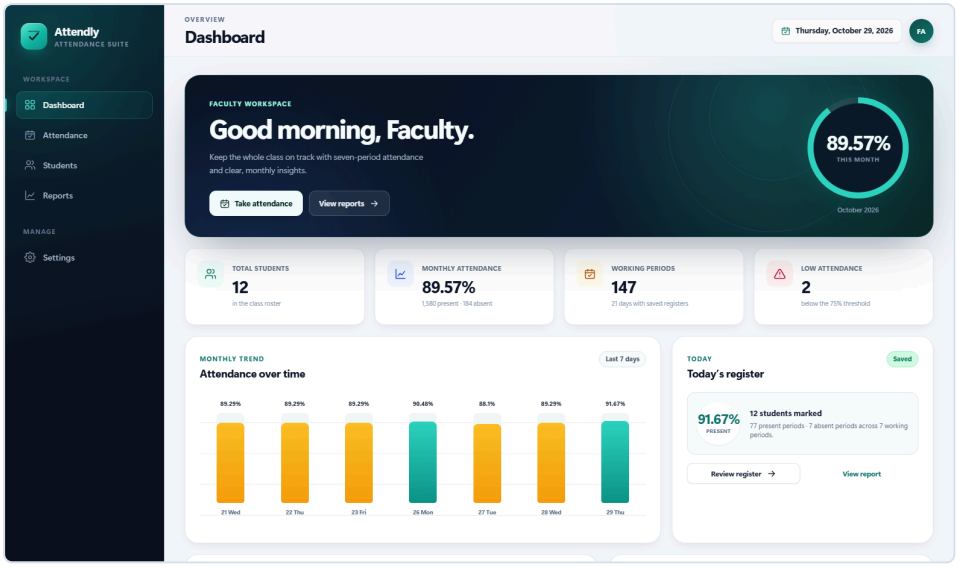


Figure 11 — Dashboard with class statistics, monthly trend, and low-attendance list.

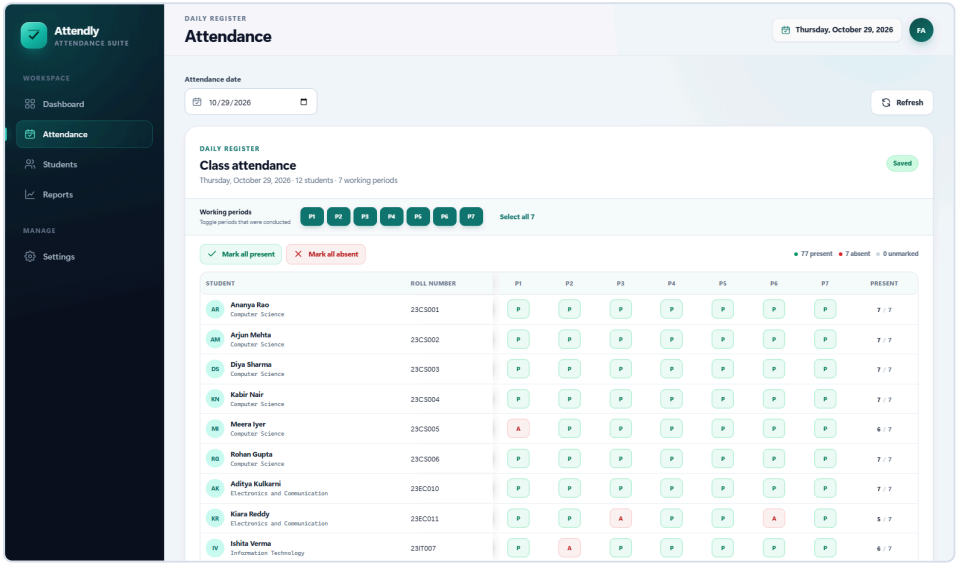


Figure 12 — Daily attendance register for seven periods, with present and absent totals.

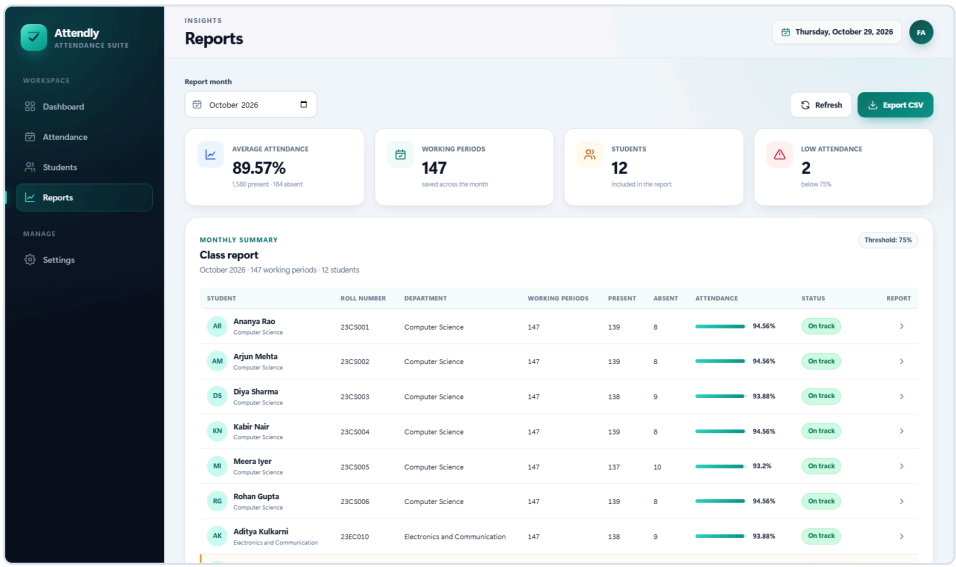


Figure 13 — Monthly class report with per-student percentages, threshold status, and CSV export.